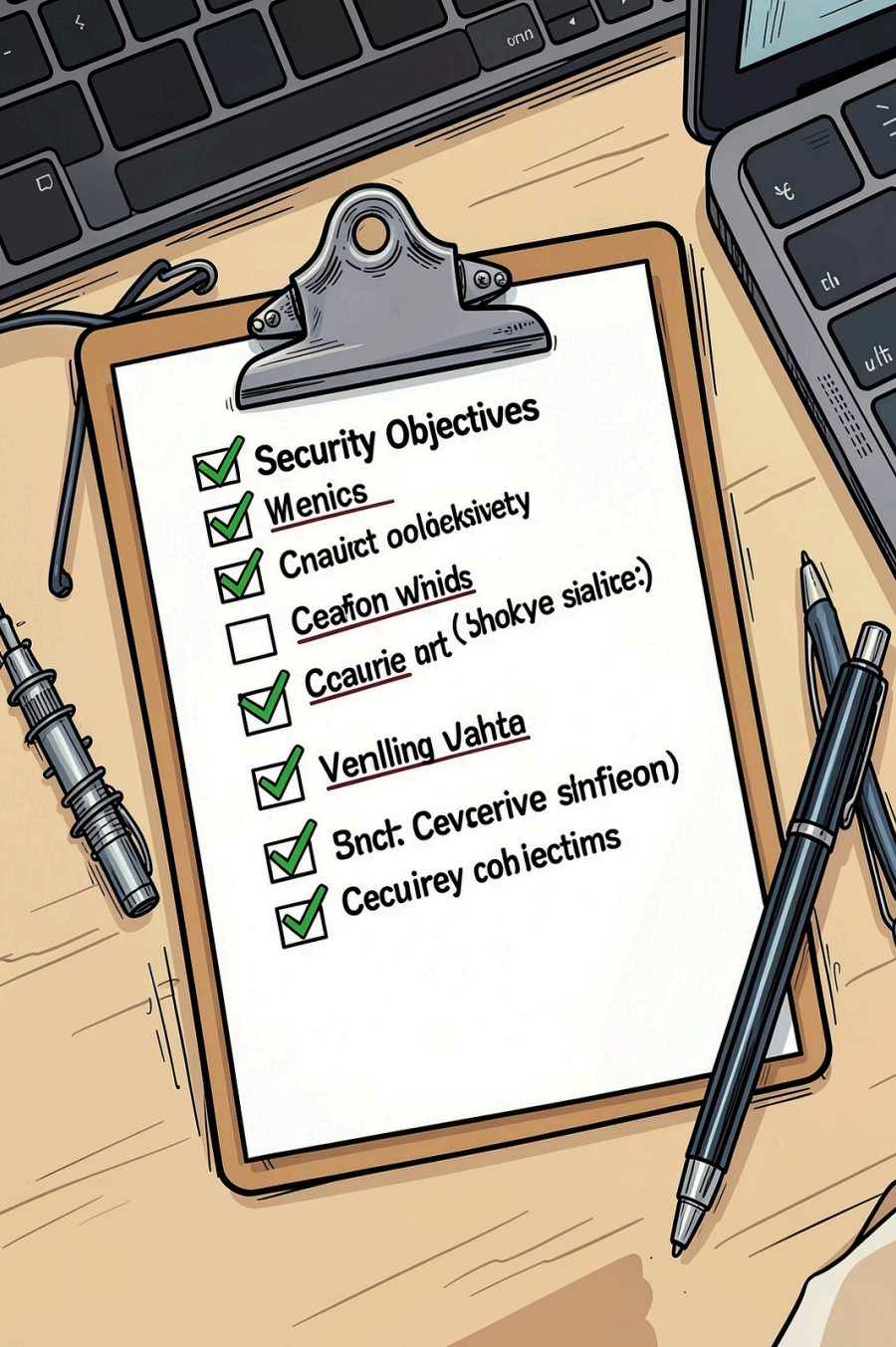


Secure Coding Review of a Vulnerable Python Login Application

This report documents a focused security assessment of a Flask-based login application. It summarizes key findings, risk ratings, and prescriptive remediation to harden the authentication codepath and support secure development practices.

1. Introduction

The scope of this internship-level review covered the authentication module and supporting persistence layer of a small Flask application backed by SQLite. The assessment applied OWASP Top 10 principles and secure coding heuristics to identify issues that could lead to data compromise, unauthorized access, or service disruption.



2. Objective

Primary Goal

Identify critical vulnerabilities in the login flow that allow authentication bypass, data leakage, or remote code execution.

Deliverables

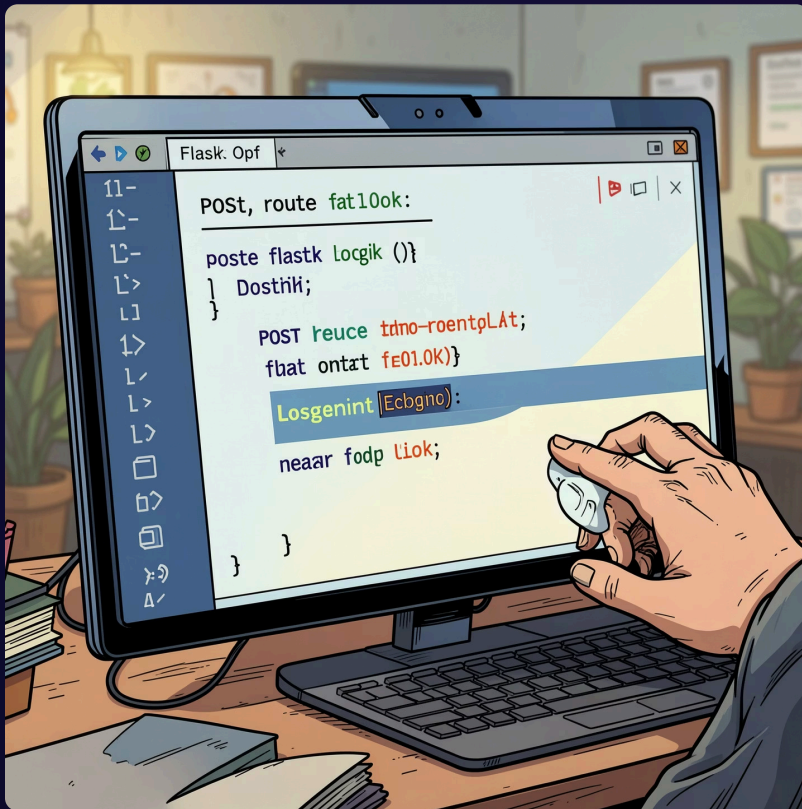
Provide prioritized findings, a risk assessment table, and clear remediation steps suitable for immediate developer action.

Constraints

Review limited to static source analysis and local runbook; no active exploitation or penetration testing was performed.

3. Source Code Overview

The application implements a POST /login endpoint that accepts username and password and queries an SQLite database to validate credentials. The code uses raw SQL string composition and opens the database connection directly in application code. Passwords appear to be compared as plain text strings.



Key components observed:

- Flask for routing and request handling
- SQLite file-based database accessed via hardcoded connection
- Direct SQL execution using string concatenation
- Minimal to no input sanitization or structured error handling

4. Vulnerability Findings

This section summarizes the high-impact vulnerabilities discovered during the code review. Each finding includes a concise description and the reason it matters to the application security posture.



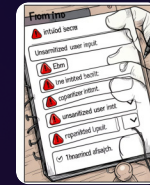
SQL Injection

User input is interpolated directly into SQL statements. An attacker can craft payloads to bypass authentication, exfiltrate user records, or modify the database.



Hardcoded Database Connection

Connection strings and credentials are embedded in source, increasing risk if the repository is shared or backups are exposed. This violates secret management best practices.



Lack of Input Validation

Username and password inputs are accepted without length checks, encoding, or canonicalization. This enables injection vectors and unexpected parsing behaviors upstream.



Poor Error Handling

Unhandled exceptions and verbose error outputs can leak schema details, SQL statements, or internal paths—information valuable to attackers performing reconnaissance.

5. Risk Assessment Table

Vulnerability	Severity	Impact
SQL Injection	Critical	Authentication bypass, data exfiltration, arbitrary DB changes
Hardcoded Secrets	High	Credential exposure, unauthorized access to DB/backups
Lack of Input Validation	Medium	Code injection, malformed requests, downstream failures
Poor Error Handling	Low	Information leakage, easier attacker reconnaissance

Risk assignments reflect probable exploitability and the magnitude of damage if abused in production.

6. Recommended Fixes



Use Parameterized Queries

Replace raw SQL with parameterized statements or an ORM (SQLAlchemy). This eliminates SQL injection by separating code and data.



Move Secrets to Environment

Store DB URIs and credentials in environment variables or a secrets manager. Rotate keys and restrict repository access.



Enforce Input Validation

Validate and canonicalize inputs server-side: length limits, allowed character sets, and encoding normalization. Reject unexpected payloads early.



Implement Secure Error Handling

Catch database errors and log minimal diagnostic details to an internal log. Return generic error messages to clients to avoid information leakage.

7. Secure Coding Best Practices



Least Privilege

Grant database accounts only the necessary permissions (SELECT/INSERT limited to required tables). Avoid using admin-level credentials in application contexts.



Password Handling

Use salted hashing (e.g., bcrypt via Werkzeug) with appropriate work factor; never store or compare plain-text passwords.



Automated Security Testing

Integrate SAST and dependency scanning into CI/CD. Block merges on high-severity findings and track remediation metrics.



Robust Logging & Monitoring

Log security-relevant events at appropriate levels, centralize logs, and configure alerts for suspicious authentication patterns.

8. Conclusion

The reviewed Flask login application contains exploitable issues—most notably SQL injection and embedded secrets—that pose substantial risk if deployed to production. The recommended fixes prioritize eliminating injection vectors, securing secrets, and strengthening validation and error management. Addressing these items will markedly improve resilience and compliance with secure development standards.

❏ Immediate priorities: parameterize queries, remove hardcoded secrets, and add server-side validation.

9. Professional Summary

This internship report underscores that proactive, code-level security reviews are a cost-effective control for preventing breaches. By adopting the recommended changes, the team will reduce attack surface, improve incident readiness, and align the project with organizational security expectations. Continued security training and automated scans should be part of the development lifecycle.

